

Advanced Lesson 11 - Exercises:

Patterns in Arguments and in Replacement Rules

Mikey Ferguson

MATH 7013 - Advanced Mathematical Modeling

1: Exactly one argument, squared

```
ClearAll[f, x, y];  
f[x_] := x2  
?  
f[2], f[x], f[y], f["string"], f[{1, 2, 3}]  
f[{1, 2}, {3, 4}]  
{f[], f[x, y], f[1, 2, 3, 4]}
```

The first six calls all match: Blank stands for exactly one expression of any kind, so a number, a symbol, a string, a list and a matrix are all acceptable. `f[{1, 2, 3}]` returns `{1, 4, 9}` and `f[{1, 2}, {3, 4}]` returns `{1, 4, 9, 16}` - not because the pattern did anything clever, but because Power is Listable and threads down through whatever structure it is handed. The last three return unevaluated. `f[]` has no argument, `f[x, y]` has two and `f[1, 2, 3, 4]` has four, and `"x_"` insists on exactly one. That is what is different about them: the count, and nothing else.

2: Exactly zero arguments

```
ClearAll[f, x, y];
```

```
f[] := 3
```

```
?f
```

```
f[]
```

```
{f[2], f[x], f[x, y], f[1, 2, 3, 4], f[{1, 2}, {3, 4}]}
```

`f[]` returns 3. All five other calls return unevaluated, and for the same reason as before: the definition was written for a call with no arguments at all, and every one of those supplies at least one. A definition with an empty argument list is not a default or a fallback; it is a pattern as strict as any other.

3: One or more arguments, squaring the list of them

```
ClearAll[f, x, y];
```

```
f[x__] := {x}^2
```

```
?f
```

```
{f[2], f[x], f[y], f["string"], f[{1, 2, 3}]}
```

```
{f[{1, 2}, {3, 4}], f[x, y], f[1, 2, 3, 4]}
```

```
f[x, {1, 2, 3}]
```

```
f[x, {1, 2, 3}, {{1, 2}, {3, 4}}]
```

```
f[]
```

`BlankSequence`, written with two underscores, matches one or more arguments and binds the name to the whole sequence. Wrapping that name in braces turns the sequence into a list, which is why `f[y]` returns `{y^2}` with the braces and `f[1, 2, 3, 4]` returns `{1, 4, 9, 16}`.

Yes, `Power` has the `Listable` attribute, and that is what makes `{x}^2` square the elements rather than attempting to square a list as an object. `Listable` means the function threads over lists automatically, one element at a time, descending as far as the structure goes - which is why `f[{1, 2, 3}]` gives `{{1, 4, 9}}`: one set of braces from the sequence, one from the argument. `f[]` is the odd one out. `BlankSequence` requires at least one argument, so a call with none does not match and comes back unevaluated.

4: Zero or more arguments

```
ClearAll[f, x, y];
f[x___] := {x}^2
?f
{f[2], f[x], f[y], f["string"], f[{1, 2, 3}]}
```

```
{f[{1, 2}, {3, 4}], f[x, y], f[1, 2, 3, 4]}
f[x, {1, 2, 3}]
f[]
```

BlankNullSequence, three underscores, is BlankSequence plus the empty case. Every result above matches Exercise 3 except the last: `f[]` now evaluates, and returns `{}`. The empty sequence wrapped in braces is the empty list, and the empty list squared is the empty list. Nothing went wrong - there was simply nothing to square.

5: Exactly two arguments

```
ClearAll[f, x, y];
f[x_, y_] := x^2 Sin[y]
?f
{f[], f[x], f[{1, 2}], f[{1, 2, 3}], f[1, 2, 3, 4]}
f[x, y]
f[x, {1, 2, 3}]
f["string", {{1, 2}, {3, 4}}]
```

`f[x, y]` returns `x^2 Sin[y]`, as intended.

The first five calls fail, and all for one reason: argument count. `f[]` has none and `f[x]` has one, so neither reaches two. `f[{1, 2}]` and `f[{1, 2, 3}]` each have exactly one argument that happens to be a list - a list is a single expression, not a sequence, and the pattern is not fooled by the comma inside the braces. `f[1, 2, 3, 4]` has four. The distinction between a list of two things and two things is the whole content of this exercise. `f[x, {1, 2, 3}]` and the string-and-matrix call do work: two arguments are two arguments, whatever they contain, and Listable Power and Sin thread into them.

6: Exactly three, and exactly four, arguments

```
ClearAll[f3, f4, x, y, z, w];
f3[x_, y_, z_] := {x, y, z};
f4[x_, y_, z_, w_] := {x, y, z, w}

{f3[1, 2, 3], f4[1, 2, 3, 4]}

{f3[1, 2], f4[1, 2, 3]}
```

Two names are used rather than one, because two definitions of the same symbol with different argument counts would coexist happily and the exercise is about the patterns rather than about overloading. Both fixed-arity patterns reject any call with the wrong number of arguments.

7: One argument, then zero or more

```
ClearAll[f, x, y, z, w, A, B];
f[x_, y___] := x^2 Sin[{y}]

?f

f[]
f[x]
f[{1, 2}]
f[x, y]
f[2, 3, 4, 5]
f[x, y, z, w]
f[x, {1, 2, 3}, A, B]
ColumnForm[f[x, {1, 2, 3}, A, B]]
f["string", {{1, 2}, {3, 4}}]
```

`f[]` fails: the first pattern requires one argument and there are none.

`f[x]` returns {}, and how Mathematica gets there is worth spelling out. The call matches with `x` bound to `x` and `y` bound to the empty sequence. The body is `x^2 Sin[{y}]`, so `Sin[{}]` is the sine of the empty list, which by listability is the empty list. Multiplying the empty list by `x^2` threads elementwise over nothing at all and returns the empty list. The `x^2` factor is not lost; there is simply nowhere to put it.

`f[{1, 2}]` does fail, but not where the exercise implies. The pattern matched perfectly well: `x_` took the whole list `{1, 2}` and `y___` took the empty sequence. The body then asks for `{1, 2}^2` times `Sin[{}]`, which is

$\{1, 4\}$ times $\{\}$. Both factors are lists, Times is Listable, and threading a two-element list against a zero-element list is impossible - so Mathematica issues Thread::tdlen, "Objects of unequal length in $\{1, 4\}$ cannot be combined", and returns the product unevaluated as $\{\}^*\{1, 4\}$. The failure is in listable multiplication, not in pattern matching, and knowing which is the whole point of asking.

$f[x, y]$ returns $\{x^2 \sin[y]\}$: one trailing argument makes a one-element list.

$f[2, 3, 4, 5]$ returns $\{4 \sin[3], 4 \sin[4], 4 \sin[5]\}$ - the leading 2 is squared once and distributed across the three sines by listability.

$f[x, \{1, 2, 3\}, A, B]$ returns a three-element list whose first entry is itself a list, because Sin threaded into the inner $\{1, 2, 3\}$. ColumnForm makes that shape legible. Mathematica calculated it by binding y to the sequence $\{1, 2, 3\}$, A , B , wrapping it in braces to get a three-element list, applying Sin elementwise, and multiplying through by x^2 elementwise.

8: Matching inside a Sine

```
ClearAll[f, x, y];
f[Sin[x_]] := x^2

?f

{f[Sin[2]], f[Sin[y]], f[Sin[Log[x]]]}

f[Sin[{1, 2, 3}]]

{f[x], f[Cos[y]], f[1, 2, 3, 4]}
```

The pattern matches an argument whose head is Sin with one part inside, and binds x to that part. So $f[\sin[2]]$ is 4, $f[\sin[y]]$ is y^2 , and $f[\sin[\log[x]]]$ is $\log[x]^2$ - the inside can be any expression at all. $f[\sin[\{1, 2, 3\}]]$ fails, and the reason is evaluation order, not pattern matching. Sin is Listable, so $\sin[\{1, 2, 3\}]$ evaluates to $\{\sin[1], \sin[2], \sin[3]\}$ before f is ever called. What f receives is a list, not a Sin, and the pattern has nothing to match. The argument was rewritten out from under the pattern.

The last three fail on shape: $f[x]$ has an argument that is not a Sin, $f[\cos[y]]$ has the wrong head, and $f[1, 2, 3, 4]$ has four arguments where the pattern wants one. That is what is different about them.

9: A derivative table by pattern matching

```
ClearAll[f, x, y];
f[Sin[x_]] := Cos[x];
f[Cos[x_]] := -Sin[x];
f[Log[x_]] :=  $\frac{1}{x}$ ;
f[e^x_] := e^x
?f

{f[e^x], f[Sin[y]], f[Cos[x + y]], f[Sin[2]]}

f[Sin[Log[x]]]

f[Sin[{1, 2, 3}]]
```

Four separate definitions on the same symbol, dispatched by the head of the argument. $f[E^x]$ is E^x , $f[\text{Sin}[y]]$ is $\text{Cos}[y]$, $f[\text{Cos}[x + y]]$ is $-\text{Sin}[x + y]$, and $f[\text{Sin}[2]]$ is $\text{Cos}[2]$.

As the exercise warns, this is not the chain rule. $f[\text{Sin}[2x]]$ would return $\text{Cos}[2x]$ rather than $2 \text{Cos}[2x]$: the pattern binds the inside of the sine and reports the outer derivative only. That is deliberate here - the exercise is about dispatch, not about differentiation, and Mathematica already has D for the real thing.

$f[\text{Sin}[\text{Log}[x]]]$ returns $\text{Cos}[\text{Log}[x]]$, again outer-only.

$f[\text{Sin}[\{1, 2, 3\}]]$ fails for exactly the reason given in Exercise 8: Sin threads over the list before f sees the argument, so no Sin remains to match.

10: Odd functions, by a recursive pattern

```
ClearAll[f, x, y];
f[-x_] := -f[x]
?f

{f[-x], f[-y], f[-Sin[y]]}

f[-5]

{FullForm[-5], FullForm[-x]}
```

$f[-x]$ returns $-f[x]$ and $f[-y]$ returns $-f[y]$, which is the odd-function identity written as a rewrite rule. $f[-\text{Sin}[y]]$ returns $-f[\text{Sin}[y]]$ for the same reason.

$f[-5]$ fails to match, and the two full forms say why. $\text{FullForm}[-x]$ is $\text{Times}[-1, x]$, a product of the literal -1 and a symbol, and the pattern $-x_$

is exactly `Times[-1, Pattern[x, Blank[]]]`, so it matches. `FullForm[-5]` is just `-5`: a single negative Integer atom, with no Times and no -1 anywhere in it. Negative numbers are stored as numbers, not as products, so there is nothing for the pattern to take apart.

11: Extending the rule to negative numbers

```
ClearAll[f, x, y];
f[-x_] := -f[x];
f[n_?Negative] := -f[-n]
?f
{f[-x], f[x], f[-y], f[-Sin[y]]}
{f[-5], f[-7], f[5]}
```

The second definition catches what the first cannot: any argument that is negative as a number, tested with the predicate rather than matched structurally. It rewrites `f[-5]` as `-f[5]`, which then has no applicable rule and stops - which is correct, since `f` is deliberately undefined on positive arguments.

Several other spellings work equally well: a pattern test with an anonymous function, or a condition attached with `"/;"`, or matching `n_Integer` and guarding on the sign. All of them share the essential move, which is to stop trying to match the minus sign and start testing the value instead.

Note that `f[x]` and `f[5]` correctly return unevaluated. An odd-function rule says how `f` behaves under negation; it says nothing about what `f` is.

12: The third element of a list of three or more

```
ClearAll[f, x, y, A, B];
f[{_, _, c_, ___}] := c
?f
{f[], f[x], f[{1, 2}]}
{f[{1, 2, 3}], f[{1, 2, 3, 4}], f[{1, 2, 3, 4, 5}]}
f[{x, y, {1, 2, 3}, A, B}]
```

The pattern is a single argument that must be a list, whose first two elements are anything, whose third is captured as `c`, and whose remaining elements - zero or more of them - are ignored.

`f[]` fails because there is no argument at all. `f[x]` fails because `x` is a symbol, not a list, so the braces in the pattern have nothing to match.

`f[{1, 2}]` fails because the list has only two elements and the pattern needs at least three - the trailing `BlankNullSequence` may be empty, but the third `Blank` may not.

The three successful calls all return 3, and the last returns `{1, 2, 3}`, which is the third element of that list whatever its own structure may be.

13: Returning the first inner list from a list

```
ClearAll[f, x, y, A, B];
f[{___, l_List, ___}] := l
?f
f[1, 2, {4, 5}, 6]
f[{1, 2, 3, 4}]
f[{1, 2, {}, 3, 4}]
{f[{1}, 2, 3, 4], f[{1, 2, {3}, 4}], f[{1, {2, 3}, 4}], f[{1, 2, 3, {4}}]}
{f[{1, 2, {3, 4, 5}}], f[{1, 2, 3, 4, 5}]}
f[{1, 2, {3, {4, 5}}}]
f[{x, y, {1, 2, 3}, A, B}]
```

The pattern is one argument which is a list containing, somewhere, an element that is itself a list; the two `BlankNullSequence`s absorb whatever lies on either side of it.

`f[1, 2, {4, 5}, 6]` fails because it supplies four arguments rather than one list. The braces in the pattern are part of the single argument, not a way of writing a sequence.

`f[{1, 2, 3, 4}]` fails because no element of that list is a list.

`f[{1, 2, {}, 3, 4}]` returns `{}`. The empty list is still a list, so the pattern matches it - the exercise's phrasing anticipates a failure, but `l_List` does not require the inner list to be non-empty. Insisting on "one or more elements", as the exercise's wording asks, needs an explicit guard such as attaching a condition that its `Length` is at least one.

The remaining calls return the inner list in each case: `{1}`, `{3}`, `{2, 3}`, `{4}`, `{3, 4, 5}`, `{1, 2, 3, 4, 5}`, and `{3, {4, 5}}`, and finally `{1, 2, 3}`.

When more than one element is a list the pattern matcher returns the first match it finds, scanning left to right.

14: Full forms of patterns, and a positive-integer function

```
Clear[x];
Column[List@@Function[u, ToString[FullForm[u]]] /@
  Hold[_ , x_ , x_ , x_ , x_Integer , x_?EvenQ , x_? (#1 > 3 &) , x_Integer?EvenQ]]
```

The eight full forms, in order:

```
Blank[]; Pattern[x, Blank[]]; Pattern[x, BlankSequence[]];
Pattern[x, BlankNullSequence[]]; Pattern[x, Blank[Integer]];
PatternTest[Pattern[x, Blank[]], EvenQ];
PatternTest[Pattern[x, Blank[]], Function[Greater[Slot[1], 3]]];
PatternTest[Pattern[x, Blank[Integer]], EvenQ].
```

The shape is uniform. Underscores are Blank, BlankSequence and BlankNullSequence; a name in front of one wraps it in Pattern; a head after one becomes Blank's argument; and a question mark wraps the whole thing in PatternTest, whose second argument is a function - a named predicate or an anonymous one, indifferently. Every piece of pattern syntax is ordinary function application wearing punctuation.

How the cell above is put together. The eight patterns are wrapped in Hold so that the sequence patterns are not spliced into the argument list of whatever encloses them. Map then works inside the Hold, producing Hold[fn[_], fn[x_], ...] without evaluating anything, and Apply replaces that Hold with List, at which point the eight ToString[FullForm[...]] calls finally evaluate and return the strings shown.

Mapping FullForm directly over the Hold is not enough: the results stay sealed inside the Hold and print as unevaluated FullForm expressions. The Apply is what releases them, and ToString is what stops the printed forms from being re-typeset back into the shorthand they came from.

```
ClearAll[f];
f[x_Integer?Positive] := "Positive Integer"

{f[3], f[3.45`], f[-3]}
```

The pattern combines both refinements at once: the head must be Integer and the value must satisfy Positive. f[3] returns the string; f[3.45] and f[-3] return unevaluated, one failing the head test and the other the predicate, which is exactly the required behaviour.

15: An immediate rule replacing 2 with 222

```
ClearAll[rule1, x];
rule1 = 2 → 222
```

```
{1, 2, 3, 4} /. rule1
```

```
Sin[2] /. rule1
```

```
{1, 2, Sin[2], x^2} /. rule1
```

```
{1, 2, 3, 2 + 2} /. rule1
```

```
{1, 2, 3, Hold[2 + 2]} /. rule1
```

```
2 Sin[2 Sin[2 Sin[2 Sin[2 x]]]] /. rule1
```

The list becomes {1, 222, 3, 4} and Sin[2] becomes Sin[222] - ReplaceAll

descends into subexpressions, so the 2 inside the sine is found. In

{1, 2, Sin[2], x^2} both the bare 2 and the one inside Sin are replaced, and so is the exponent of x^2, since an exponent is an ordinary part of a Power expression.

The fourth list is the interesting one. It returns {1, 222, 3, 4}: the last entry was missed. Nothing went wrong with the rule - 2 + 2 was evaluated to 4 before ReplaceAll ever ran, so by the time the rule was applied there was no 2 left to find. Rules act on expressions as they are, not as they were typed.

Wrapping it in Hold proves the point: Hold[2 + 2] survives evaluation, the rule reaches inside it, and the result is Hold[222 + 222].

The nested sine becomes 222 Sin[222 Sin[222 Sin[222 Sin[222 x]]]] - every occurrence at every depth, in one pass.

16: An immediate rule on a cube

```
ClearAll[rule2, x, y, z];
```

```
rule2 = x_3 → x333
```

```
{x2, x3, x4, y3, Sin[x]3,  $\frac{1}{2} + z^3$ , (x + y)3} /. rule2
```

```
 $\left(\frac{1}{2} + y\right)^3$  /. rule2
```

```
{FullForm[x3], FullForm[(2 + y)3], FullForm[ $\left(\frac{1}{2} + y\right)^3$ ]}
```

```
Hold[ $\left(\frac{1}{2} + y\right)^3$ ] /. rule2
```

In the list, x^3, y^3, Sin[x]^3 and (x + y)^3 are all rewritten - the base may be any expression - while x^2 and x^4 are untouched because the exponent must be exactly 3. The z^3 inside the sum is rewritten too.

(1/2 + y)^3 on its own is also rewritten, to (1/2 + y)^333. The exercise expects this one to be missed, and in the printed version it presumably was;

in this version of the language it is not. The full forms explain the distinction the exercise is pointing at: x^3 is `Power[x, 3]` and $(2 + y)^3$ is `Power[Plus[2, y], 3]`, both with a literal 3 in the exponent, so both match. The `Hold` version confirms that the rule is matching the exponent and not something the evaluator produced afterwards.

17: Two rules in a list, and why the first one gets missed

```
{x^2, x^3, x^4, y^3, Sin[x]^3,  $\frac{1}{2} + z^3$ , (x + y)^3} /. {rule1, rule2}
```

```
(2 + y)^3 /. {rule1, rule2}
```

```
(2 + y)^3 //. {rule1, rule2}
```

Applying both rules to $(2 + y)^3$ with `ReplaceAll` gives $(222 + y)^{333}$, but the order in which that happens repays attention. `ReplaceAll` works top down: at the outermost position the whole expression matches rule2 and is rewritten to $(2 + y)^{333}$, and `ReplaceAll` does not re-examine what it just produced - so at the top level rule1 never fires. It fires only where it is reached independently, on the 2 inside the base, during the same single pass. `ReplaceRepeated` re-scans until nothing changes. That is why `///.` catches everything - including, here, the 333 itself, which contains no 2 and so survives, but in general repeated replacement on rules that can regenerate their own left-hand sides will not terminate. Use it deliberately.

18: Turning a plotted curve into a cloud of points

```
ClearAll[x, plot1, plot2, plot3];
```

```
plot1 = Plot[x^2, {x, 0, 1}]
```

```
Length[Cases[plot1, _Line, ∞]]
```

A plot is not a picture; it is a `Graphics` expression containing directives and primitives. Inside this one there is a single `Line` primitive holding the whole sampled curve as a list of coordinate pairs.

```
plot2 = plot1 /. Line[{pts__}] -> Line[Point /@ {pts}];
```

```
Length[Cases[plot2, _Point, ∞]]
```

Step one. The rule matches a `Line` holding a sequence of coordinate pairs, captures that sequence, and rebuilds a `Line` whose contents are those pairs each wrapped in `Point`. The result is not a valid graphic - a `Line` is not allowed to contain `Point` primitives - which is why the exercise says plot2 cannot be displayed and why the cell ends in a semicolon. It is an

intermediate form, and the count confirms it holds one Point for every sample the plot took.

A delayed rule is used because the right-hand side calls Map on the captured sequence, and that work must happen after the match, not before.

```
plot3 = plot2 /. Line[{pts__Point}] => {pts};
{Head[plot3], Length[Cases[plot3, _Line, ∞]], Length[Cases[plot3, _Point, ∞]]}

plot3
```

Step two. The second rule strips the now-meaningless Line wrapper and leaves the bare list of Point primitives, which is a perfectly legal graphic. The counts confirm it: no Line objects remain, and every Point survived. plot3 displays as the same curve rendered as a dotted trace.

The conclusion the exercise is driving at is the right one. A finished graphic is an expression like any other, and the same pattern matching used on algebra works on it unchanged. There is no special graphics-editing facility because none is needed.